

PocketLedger

A Personal Expense Tracker



GROUP 02
BECS 31242 - VISUAL PROGRAMMING

T.M.D.P.M. De Alwis	- EC/2022/012
P.M.R.N.D.B. Wathurakumbura	- EC/2022/056
K.P. Tharuka	- PS/2022/205
W.P.M. Manavi	- EC/2022/062

1. Introduction

PocketLedger is a desktop personal finance application built in C# (.NET, Windows Forms) with a MySQL database back end. It was developed to give individuals, particularly students and professionals, a simple, focused way to track day-to-day income and expenses without the complexity of full-scale enterprise accounting software.

The application runs entirely on the user's own machine, keeping financial data local rather than relying on a cloud service. Each user has a private account, and every feature in the system - transaction logging, budgeting, and reporting - is scoped to that account so personal financial data stays isolated and secure.

Core Capabilities

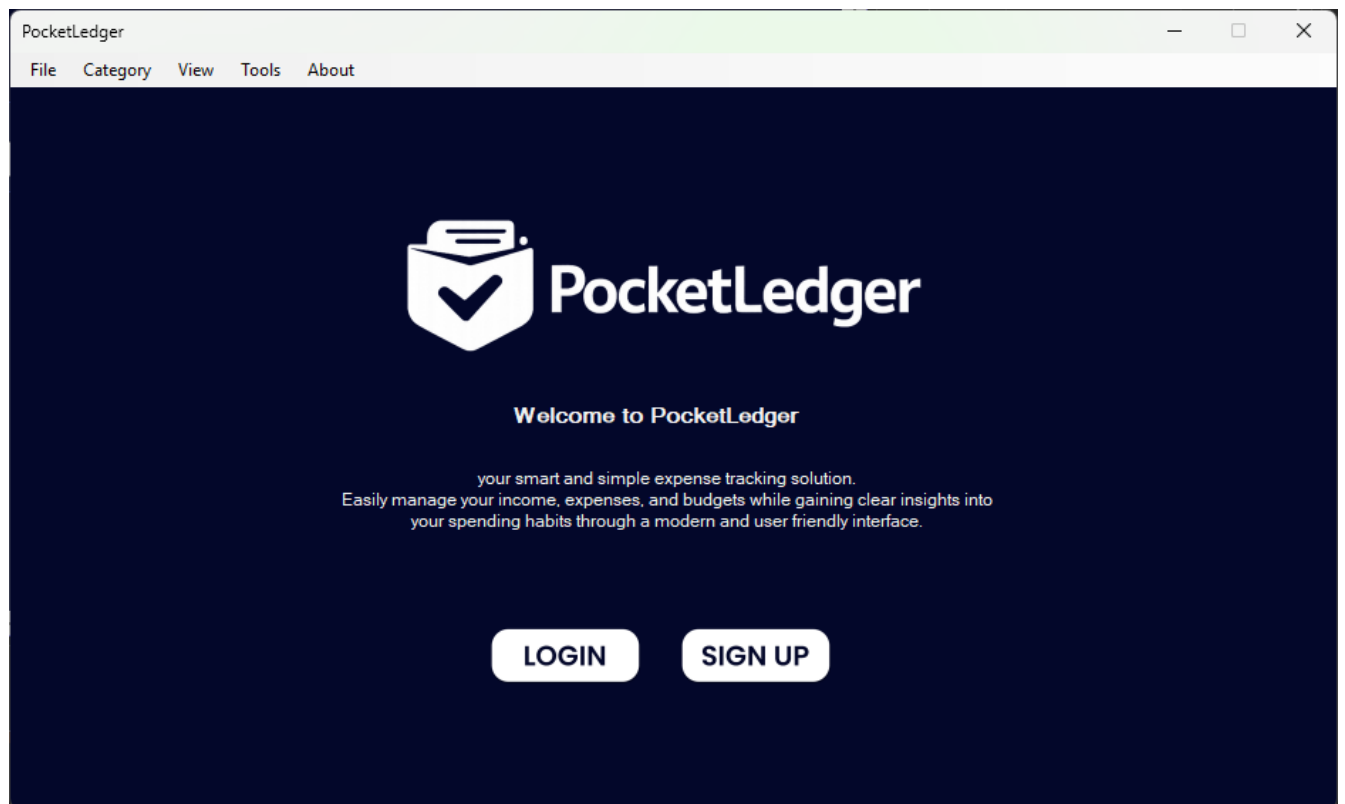
- Secure account creation and login, with profile picture support.
- Logging of income and expenses, categorized and tagged with a payment method, date, and optional notes.
- A combined transaction history with filtering and search.
- Category-based monthly budgeting with live status tracking (Safe / Warning / Over Budget).
- Visual financial reports summarizing spending by category, income vs. expense, and trends over time

2. Main Functionalities

This section walks through each screen of the application and what it does. A placeholder has been left under each heading for the corresponding interface screenshot.

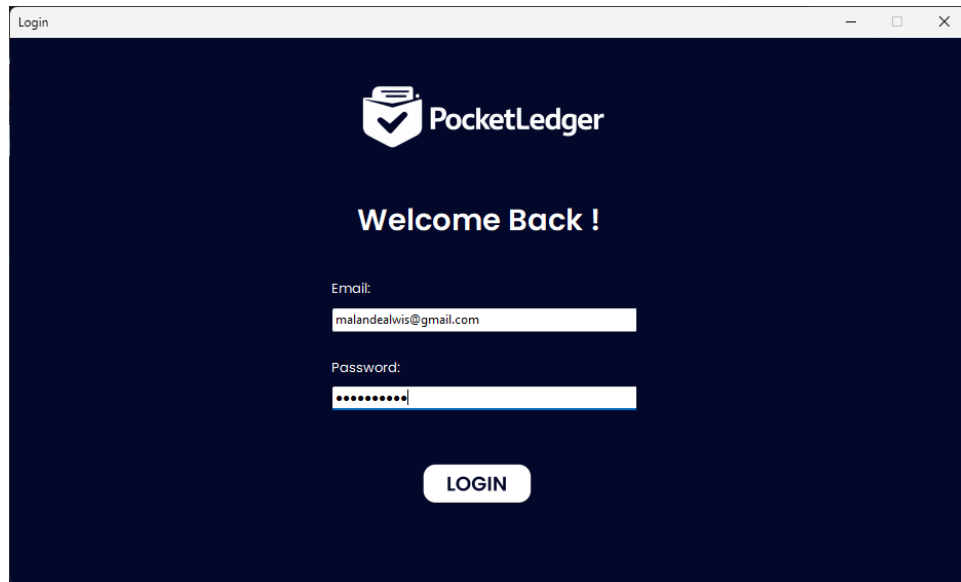
2.1 Landing Page

The entry screen shown when the application launches. It presents the PocketLedger branding and routes the user to either the Login or Sign Up screen before any financial data is accessed.



2.2 Login

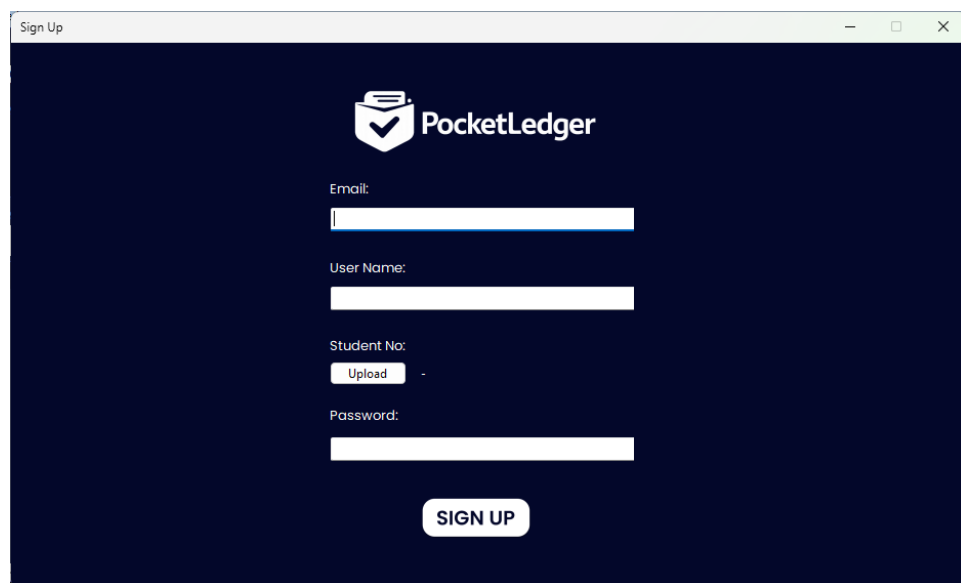
Allows a registered user to sign in using their email and password. Credentials are validated against the Users table through `UserRepository.Login()`, and a successful login opens the Dashboard for that specific user's account.



The screenshot shows a web browser window titled "Login". The background is dark blue. At the top center is the PocketLedger logo, which consists of a white shield icon with a checkmark inside, followed by the text "PocketLedger" in white. Below the logo, the text "Welcome Back !" is displayed in white. Underneath, there are two input fields: "Email:" with the value "malandealwis@gmail.com" and "Password:" with a masked password represented by dots. A white "LOGIN" button is centered at the bottom of the form area.

2.3 Sign Up (Registration)

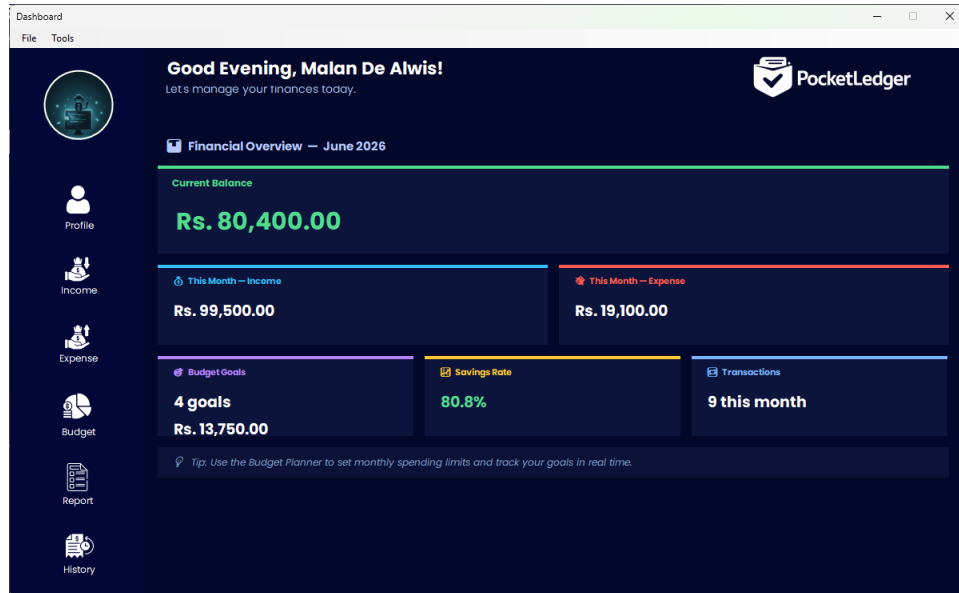
Allows a new user to create an account by entering a username, email, and password, and uploading a profile picture (via a file-browser dialog). Input is validated before the account is created through `UserRepository.Signup()`, after which the user can log in.



The screenshot shows a web browser window titled "Sign Up". The background is dark blue. At the top center is the PocketLedger logo, which consists of a white shield icon with a checkmark inside, followed by the text "PocketLedger" in white. Below the logo, there are four input fields: "Email:", "User Name:", "Student No:" (with an "Upload" button next to it), and "Password:". A white "SIGN UP" button is centered at the bottom of the form area.

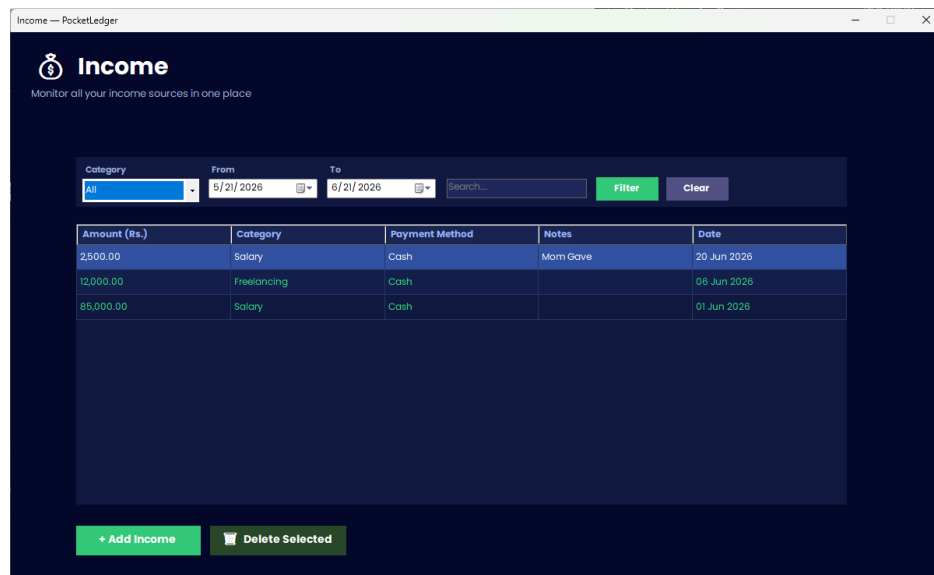
2.4 Dashboard

The main interface after login. It greets the user by name with a time-of-day-aware message (Good Morning/Afternoon/Evening), displays a circular profile picture, and shows a live financial overview made up of summary cards: Current Balance, This Month's Income, This Month's Expense, Budget Goals, Savings Rate, and Transaction Count - all pulled from DashboardRepository. A sidebar gives one-click navigation to every other module (Income, Expenses, History, Budget Planner, Reports, Profile).



2.5 Add Income

A dedicated screen for recording income entries. Users choose a category (e.g. Salary, Business, Freelance), payment method, date, amount, and an optional note. Existing income records can be filtered by category, date range, or keyword search, and summary cards show the total, highest, and average income for the period. Records can also be deleted.



Add Income — PocketLedger

Amount (Rs.)

Category

Freelancing

Payment Method

Bank Transfer

Date

6/21/2026

Notes (optional)

Add a note about this income...

Save Income

Cancel

2.6 Add Expense

Mirrors the Income screen but for spending. Users log expenses against categories such as Food & Dining, Transport, Bills & Utilities, with payment method, date, amount, and notes. The same filter/search and summary-card pattern (total, highest, average) is available, along with the ability to delete an entry.

Expenses — PocketLedger

Expenses

Track and manage all your expense transactions

Category

All

From

5/21/2026

To

6/21/2026

Search...

Filter

Clear

Amount (Rs.)	Category	Payment Method	Notes	Date
5,600.00	Groceries	Cash	Keells	20 Jun 2026
1,800.00	Entertainment	Cash		10 Jun 2026
3,200.00	Bills & Utilities	Cash		08 Jun 2026
2,500.00	Transport	Cash		05 Jun 2026
4,500.00	Groceries	Cash		03 Jun 2026
1,500.00	Food	Cash		02 Jun 2026

+ Add Expense

Delete Selected

Add Expense — PocketLedger

Amount (Rs.)

Category

Bills & Utilities

Payment Method

Cash

Date

6/21/2026

Notes (optional)

Add a note about this expense...

Save Expense

Cancel

2.7 Transaction History

A consolidated, read-focused view that merges income and expense records into a single list. Users can filter by transaction type, category, date range, and free-text search. Rows are colour-coded (green for income, red for expense) so spending patterns are easy to understand.

History

File Tools

Transaction History

PocketLedger

ID	Type	Amount	Category	Description	Date
78	Expense	5600.00	Groceries	Keells	6/20/2026
77	Income	2500.00	Salary	Mom Gave	6/20/2026
67	Expense	1800.00	Entertainment	Movie tickets	6/10/2026
66	Expense	3200.00	Bills & Utilities	Electricity bill	6/8/2026
65	Income	12000.00	Freelancing	Freelance web project	6/6/2026
64	Expense	2500.00	Transport	Fuel expenses	6/5/2026
63	Expense	4500.00	Groceries	Weekly groceries	6/3/2026
62	Expense	1500.00	Food	Lunch and snacks	6/2/2026
61	Income	85000.00	Salary	Monthly Salary	6/1/2026

Search:

From:

6/21/2026

To:

6/21/2026

Category:

All

Type:

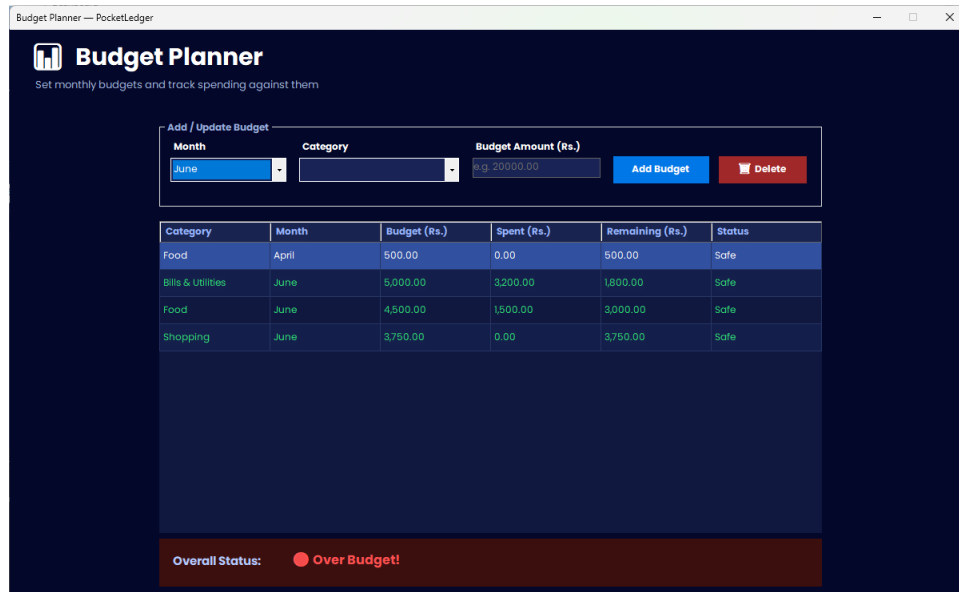
All

Filter

Clear

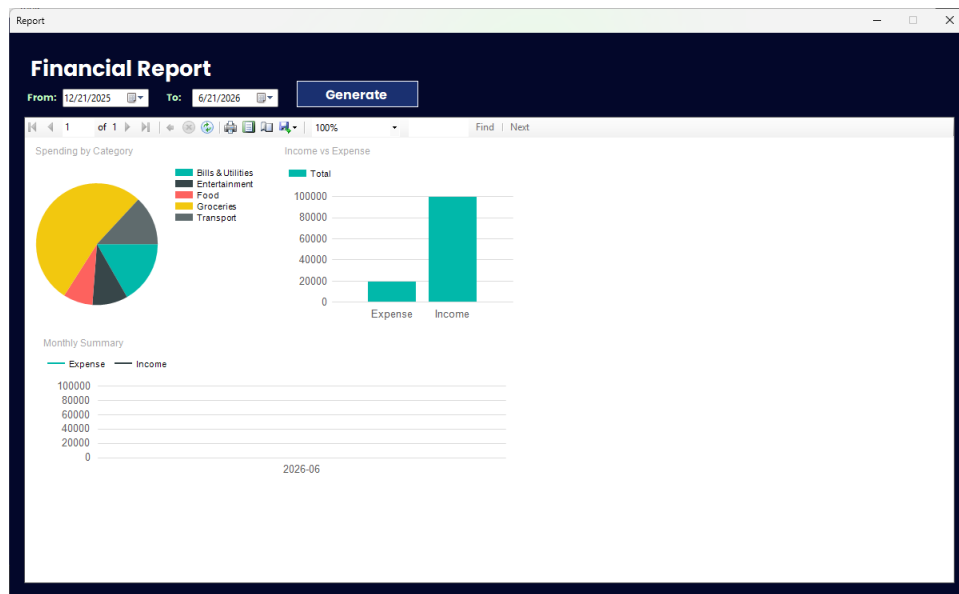
2.8 Budget Planner

Allows users to set a monthly spending limit per category. Each budget is tracked live against actual spending using a database view (BudgetSummaryView), and is automatically labelled Safe, Warning (70% utilised), or Over Budget, with colour-coded rows and an overall summary panel showing total budget, total spent, and remaining balance.



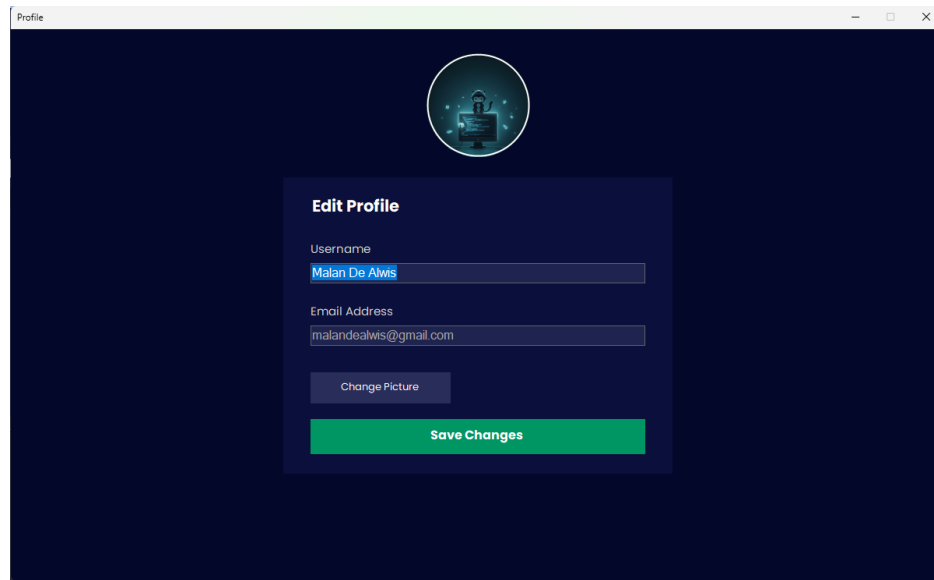
2.9 Reports

Generates a financial report (built with Microsoft's ReportViewer/RDLC engine) over a chosen date range, combining three views: spending broken down by category, an income-vs-expense comparison, and a month-by-month summary, giving the user a higher-level picture of their finances over time.



2.10 Profile

Allows the logged-in user to view and update their account details and change their display name and profile picture.



3. Contribution of Each Group Member

#	Member Name	Student No.	Assigned Forms
1	T.M.D.P.M De Alwis	EC/2022/012	Login Form, Registration Form, Dashboard Form, Landing Form
2	P.M.R.N.D.B Wathurakumbura	EC/2022/056	Add Income Form, Add Expense Form
3	K.P Tharuka	PS/2022/205	User Profile Form, Budget Planner Form
4	W.P.M Manavi (Movini)	EC/2022/062	Transaction History Form, Reports Form

3.1 T.M.D.P.M De Alwis (Malan) (EC/2022/012)

Assigned forms: Landing Form, Login Form, Registration Form, Dashboard Form

- Built the Dashboard, the application's main hub - including the time-aware greeting, circular profile picture rendering, the sidebar navigation linking to all other modules, and the live financial summary cards (balance, income, expense, budget goals, savings rate, transaction count) powered by (DashboardRepository.cs)
- Built the Login screen, including the email/password input fields and the handoff to (UserRepository.Login) for credential verification.
- Built the Sign Up (Registration) screen, including profile-picture upload (file browser dialog), field validation, and account creation through (UserRepository.Signup)
- Structured the application's codebase - organizing all user interface screens and presentation files in the Forms folder, and all query execution, connection, and data fetching operations in the Database folder.
- Implemented password security and login verification - using BCrypt inside (UserRepository.cs) to securely hash passwords during signup and authenticate users during login.

3.2 P.M.R.N.D.B Wathurakumbura (Nadula) (EC/2022/056)

Assigned forms: Add Income Form, Add Expense Form

- Built the Add Income screen - category/payment-method selection, date entry, notes, and a filterable, searchable list of income records with total/highest/average summary cards.
- Built the Add Expense screen, mirroring the Income workflow for expense entries, including category breakdown and the same filtering, search, and summary-card features.
- Implemented the underlying add/delete transaction logic shared by both screens through IncomeExpenseRepository.

3.3 K.P Tharuka (PS/2022/205)

Assigned forms: User Profile Form, Budget Planner Form

- Built the Budget Planner module, allowing users to set monthly budgets per category and tracking live status (Safe / Warning / Over Budget) against actual spend using the BudgetSummaryView database view, plus an overall budget summary panel.
- Built the User Profile screen, allowing the logged-in user to update their display name and profile picture, saved via UserRepository.UpdateUserProfile().

3.4 W.P.M Manavi (Movini) (EC/2022/062)

Assigned forms: Transaction History Form, Reports Form

- Built the Transaction History screen - a combined view of income and expense records with filtering by type, category, and date range, free-text search, and colour-coded rows by transaction type.
- Built the Reports module, generating an RDLC-based financial report (category breakdown, income-vs-expense comparison, and monthly summary) using Microsoft ReportViewer.
- Implemented and TransactionRepository.cs to centralise transaction database queries, and fixed a data-isolation bug so each user only sees their own records.